COMP2111 Week 10
Term 1, 2023
Lambda Calculus and Higher-Order Logic

Week 10

Today λ-calculus.

Friday Wrapup and exam prep.

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Final exam

- The exam is a 24-hour take-home exam.

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Final exam

- The exam is a 24-hour take-home exam.
- Wednesday May 3rd 8AM–Friday May 4th 8AM.

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Final exam

- The exam is a 24-hour take-home exam.
- Wednesday May 3rd 8AM–Friday May 4th 8AM.
- When the time starts, you will receive the exam questions in your UNSW inbox.

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Final exam

- The exam is a 24-hour take-home exam.
- Wednesday May 3rd 8AM–Friday May 4th 8AM.
- When the time starts, you will receive the exam questions in your UNSW inbox.
- Submission via pdf through give.

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Final exam

- The exam is a 24-hour take-home exam.
- Wednesday May 3rd 8AM–Friday May 4th 8AM.
- When the time starts, you will receive the exam questions in your UNSW inbox.
- Submission via pdf through give.
- You can ask questions on Ed during the exam. I sleep at night, so try to ask early.

Admin announcements

myExperience

- The myExperience survey is up. Link on the website. Please take a moment to fill it in, it helps a lot! :)

Final exam

- The exam is a 24-hour take-home exam.
- Wednesday May 3rd 8AM–Friday May 4th 8AM.
- When the time starts, you will receive the exam questions in your UNSW inbox.
- Submission via pdf through give.
- You can ask questions on Ed during the exam. I sleep at night, so try to ask early.
- The intended workload is < 4 hours; the 24 hours are just to give you flexibility. We can't stop you from using more than 4 hours...

Detour: program evaluation

```
int f (int n) {
  if n = 0 then 1 else 0
}
```

Let's evaluate this function step by step:

$f(0) \longrightarrow$

Detour: program evaluation

```
int f (int n) {
  if n = 0 then 1 else 0
}
```

Let's evaluate this function step by step:

$$
\begin{array}{ll}
f(0) & \longrightarrow \text{(function application)} \\
\texttt{if } 0 = 0 \texttt{ then } 1 \texttt{ else } 0 & \longrightarrow
\end{array}
$$

Function application: substitute the argument into the body.

Detour: program evaluation

```
int f (int n) {
  if n = 0 then 1 else 0
}
```

Let's evaluate this function step by step:

$$
\begin{array}{ll}
f(0) & \longrightarrow \text{(function application)} \\
\texttt{if } 0 = 0 \texttt{ then } 1 \texttt{ else } 0 & \longrightarrow \text{(equality comparison)} \\
\texttt{if true then } 1 \texttt{ else } 0 & \longrightarrow
\end{array}
$$

Function application: substitute the argument into the body.

Detour: program evaluation

```
int f (int n) {
  if n = 0 then 1 else 0
}
```

Let's evaluate this function step by step:

$$
\begin{array}{ll}
f(0) & \longrightarrow \text{(function application)} \\
\texttt{if } 0 = 0 \texttt{ then } 1 \texttt{ else } 0 & \longrightarrow \text{(equality comparison)} \\
\texttt{if true then } 1 \texttt{ else } 0 & \longrightarrow \text{(if true)} \\
1 &
\end{array}
$$

Function application: substitute the argument into the body.

Detour: program evaluation

The λ-calculus formalises this kind of step-by-step evaluation of program expressions...

Detour: program evaluation

The λ-calculus formalises this kind of step-by-step evaluation of program expressions...

...for a tiny language where the *only* operation is function application. Nonetheless, λ-calculus is Turing complete. How can that be?

λ-calculus

Alonzo Church

- lived 1903–1995
- supervised people like Alan Turing, Stephen Kleene
- famous for Church-Turing thesis, lambda calculus,
  first undecidability results
- invented λ calculus in 1930's

λ-calculus

Alonzo Church

- lived 1903–1995
- supervised people like Alan Turing, Stephen Kleene
- famous for Church-Turing thesis, lambda calculus,
  first undecidability results
- invented λ calculus in 1930's

λ-calculus

- originally meant as foundation of mathematics
- important applications in theoretical computer science
- foundation of computability and functional programming

untyped λ-calculus

Basic intuition:

instead of	$f(x) = x + 5$
write	$f = \lambda x.\ x + 5$

untyped λ-calculus

Basic intuition:

instead of	$f(x) = x + 5$
write	$f = \lambda x.\ x + 5$

- a term

untyped λ-calculus

Basic intuition:

instead of	$f(x) = x + 5$
write	$f = \lambda x.\ x + 5$

$\lambda x.\ x + 5$

- a term
- a nameless function

untyped λ-calculus

Basic intuition:

instead of	$f(x) = x + 5$
write	$f = \lambda x.\ x + 5$

$\lambda x.\ x + 5$

- a term
- a nameless function
- that adds 5 to its parameter

Function Application

For applying arguments to functions

instead of	$f(a)$
write	$f\ a$

Function Application

For applying arguments to functions

instead of	$f(a)$
write	$f\ a$

Example: $(\lambda x.\ x + 5)\ a$

Function Application

For applying arguments to functions

instead of	$f(a)$
write	$f\ a$

Example: $(\lambda x.\ x + 5)\ a$

Evaluating: in $(\lambda x.\ t)\ a$ replace x by a in t
(computation!)

Function Application

For applying arguments to functions

instead of $f(a)$
write $f\ a$

Example: $(\lambda x.\ x + 5)\ a$

Evaluating: in $(\lambda x.\ t)\ a$ replace x by a in t
(computation!)

Example: $(\lambda x.\ x + 5)\ (a + b)$ evaluates to $(a + b) + 5$

That's it!

That's it!

Now Formally

Syntax

Terms: $t \quad ::= \quad v \mid c \mid (t\ t) \mid (\lambda x.\ t)$

$v, x \in V, \quad c \in C, \quad V, C$ sets of names

Syntax

Terms: $t \quad ::= \quad v \mid c \mid (t\ t) \mid (\lambda x.\ t)$

$v, x \in V, \quad c \in C, \quad V, C$ sets of names

- v, x variables
- c constants
- $(t\ t)$ application
- $(\lambda x.\ t)$ abstraction

Conventions

- leave out parentheses where possible
- list variables instead of multiple λ

Example: instead of $(\lambda y.\ (\lambda x.\ (x\ y)))$ write $\lambda y\ x.\ x\ y$

Conventions

- leave out parentheses where possible
- list variables instead of multiple λ

Example: instead of $(\lambda y.\ (\lambda x.\ (x\ y)))$ write $\lambda y\ x.\ x\ y$

Rules:

- list variables: $\lambda x.\ (\lambda y.\ t) = \lambda x\ y.\ t$
- application binds to the left: $x\ y\ z = (x\ y)\ z \neq x\ (y\ z)$
- abstraction binds to the right:
  $\lambda x.\ x\ y = \lambda x.\ (x\ y) \neq (\lambda x.\ x)\ y$
- leave out outermost parentheses

Getting used to the Syntax

Example:

$\lambda x\ y\ z.\ x\ z\ (y\ z) =$

Getting used to the Syntax

Example:

$\lambda x\ y\ z.\ x\ z\ (y\ z) =$

$\lambda x\ y\ z.\ (x\ z)\ (y\ z) =$

Getting used to the Syntax

Example:

$\lambda x\ y\ z.\ x\ z\ (y\ z) =$

$\lambda x\ y\ z.\ (x\ z)\ (y\ z) =$

$\lambda x\ y\ z.\ ((x\ z)\ (y\ z)) =$

Getting used to the Syntax

Example:

$\lambda x\ y\ z.\ x\ z\ (y\ z) =$

$\lambda x\ y\ z.\ (x\ z)\ (y\ z) =$

$\lambda x\ y\ z.\ ((x\ z)\ (y\ z)) =$

$\lambda x.\ \lambda y.\ \lambda z.\ ((x\ z)\ (y\ z)) =$

Getting used to the Syntax

Example:

$\lambda x\ y\ z.\ x\ z\ (y\ z) =$

$\lambda x\ y\ z.\ (x\ z)\ (y\ z) =$

$\lambda x\ y\ z.\ ((x\ z)\ (y\ z)) =$

$\lambda x.\ \lambda y.\ \lambda z.\ ((x\ z)\ (y\ z)) =$

$(\lambda x.\ (\lambda y.\ (\lambda z.\ ((x\ z)\ (y\ z)))))$

Encoding numbers

Remember Peano arithmetic?

$$\begin{aligned} 0 &\equiv Z \\ 1 &\equiv S(Z) \\ 2 &\equiv S(S(Z)) \end{aligned}$$

In the λ-calculus we have *Church numerals*:

$$\begin{aligned} 0 &\equiv \lambda f\, x.\, x \\ 1 &\equiv \lambda f\, x.\, f\ x \\ 2 &\equiv \lambda f\, x.\, f(f\ x) \end{aligned}$$

Encoding numbers

$$
\begin{aligned}
0 &\equiv \lambda f\, x.\, x \\
1 &\equiv \lambda f\, x.\, f\ x \\
2 &\equiv \lambda f\, x.\, f(f\ x)
\end{aligned}
$$

- We encode a number n a function...

Encoding numbers

$$
\begin{aligned}
0 &\equiv \lambda f\, x.\, x \\
1 &\equiv \lambda f\, x.\, f\ x \\
2 &\equiv \lambda f\, x.\, f(f\ x)
\end{aligned}
$$

- We encode a number n a function...
- ..that accepts two arguments, f and x...

Encoding numbers

$$
\begin{aligned}
0 &\equiv \lambda f\, x.\, x \\
1 &\equiv \lambda f\, x.\, f\ x \\
2 &\equiv \lambda f\, x.\, f(f\ x)
\end{aligned}
$$

- We encode a number n a function...
- ..that accepts two arguments, f and x...
- ..and returns the result of applying f to x n times.

Encoding numbers

$$
\begin{aligned}
0 &\equiv \lambda f\, x.\, x \\
1 &\equiv \lambda f\, x.\, f\ x \\
2 &\equiv \lambda f\, x.\, f(f\ x)
\end{aligned}
$$

Here's the successor function (given m return $m + 1$):

$$\lambda m.\, \lambda f\, x.\, f\ (m\ f\ x)$$

Encoding numbers

$$
\begin{aligned}
0 &\equiv \lambda f\, x.\, x \\
1 &\equiv \lambda f\, x.\, f\ x \\
2 &\equiv \lambda f\, x.\, f(f\ x)
\end{aligned}
$$

Here's addition (given m and n return $m + 1$):

$$\lambda m\, n.\, \lambda f\, x.\, m\ f\ (n\ f\ x)$$

Encoding booleans

$$\begin{array}{lcl} \texttt{true} & \equiv & \lambda x\ y.\ x \\ \texttt{false} & \equiv & \lambda x\ y.\ y \end{array}$$

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Encoding booleans

$$
\begin{array}{lcl}
\texttt{true} & \equiv & \lambda x\ y.\ x \\
\texttt{false} & \equiv & \lambda x\ y.\ y \\
\neg b & \equiv &
\end{array}
$$

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Encoding booleans

$$
\begin{array}{lcl}
\texttt{true} & \equiv & \lambda x\ y.\ x \\
\texttt{false} & \equiv & \lambda x\ y.\ y \\
\neg b & \equiv & \lambda b.\ b\ \texttt{false}\ \texttt{true}
\end{array}
$$

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Encoding booleans

$$
\begin{array}{lcl}
\texttt{true} & \equiv & \lambda x\ y.\ x \\
\texttt{false} & \equiv & \lambda x\ y.\ y \\
\neg b & \equiv & \lambda b.\ b\ \texttt{false}\ \texttt{true} \\
\texttt{if}\ z\ \texttt{then}\ x\ \texttt{else}\ y & \equiv &
\end{array}
$$

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Encoding booleans

$$
\begin{array}{lcl}
\texttt{true} & \equiv & \lambda x\ y.\ x \\
\texttt{false} & \equiv & \lambda x\ y.\ y \\
\neg b & \equiv & \lambda b.\ b\ \texttt{false}\ \texttt{true} \\
\texttt{if}\ z\ \texttt{then}\ x\ \texttt{else}\ y & \equiv & \lambda z\ x\ y.\ z\ x\ y
\end{array}
$$

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Encoding booleans

$$
\begin{array}{lcl}
\texttt{true} & \equiv & \lambda x\ y.\ x \\
\texttt{false} & \equiv & \lambda x\ y.\ y \\
\neg b & \equiv & \lambda b.\ b\ \texttt{false}\ \texttt{true} \\
\texttt{if}\ z\ \texttt{then}\ x\ \texttt{else}\ y & \equiv & \lambda z\ x\ y.\ z\ x\ y \\
\texttt{isZero}(n) & \equiv &
\end{array}
$$

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Encoding booleans

$$
\begin{array}{lcl}
\texttt{true} & \equiv & \lambda x\ y.\ x \\
\texttt{false} & \equiv & \lambda x\ y.\ y \\
\neg b & \equiv & \lambda b.\ b\ \texttt{false}\ \texttt{true} \\
\texttt{if}\ z\ \texttt{then}\ x\ \texttt{else}\ y & \equiv & \lambda z\ x\ y.\ z\ x\ y \\
\texttt{isZero}(n) & \equiv & \lambda n.\ n\ (\lambda x.\ \texttt{false})\ \texttt{true}
\end{array}
$$

We encode booleans as binary functions. `true` returns the first argument, `false` returns the second argument.

Detour revisited

$$
\begin{array}{l}
\texttt{int}\ f\ (\texttt{int}\ n)\ \{ \\
\quad \texttt{if}\ n = 0\ \texttt{then}\ 1\ \texttt{else}\ 0 \\
\}
\end{array}
$$

This program can be encoded in λ-calculus as follows:

$$
\begin{array}{l}
\lambda n. \\
\quad (n\ (\lambda x.\ (\lambda x\ y.\ y))\ (\lambda x\ y.\ x)) \\
\qquad (\lambda f\ x.\ f\ x) \\
\qquad (\lambda f\ x.\ x)
\end{array}
$$

Detour revisited

$$
\begin{array}{l}
\texttt{int } f\ (\texttt{int } n)\ \{ \\
\quad \texttt{if } n = 0 \texttt{ then } 1 \texttt{ else } 0 \\
\}
\end{array}
$$

This program can be encoded in λ-calculus as follows:

$$
\begin{array}{l}
\lambda n. \\
\quad (n\ (\lambda x.\ (\lambda x\ y.\ y))\ (\lambda x\ y.\ x)) \quad \texttt{if isZero}(n)\ \texttt{then} \\
\qquad (\lambda f\ x.\ f\ x) \\
\qquad (\lambda f\ x.\ x)
\end{array}
$$

Detour revisited

$$
\begin{array}{l}
\texttt{int } f\ (\texttt{int } n)\ \{ \\
\quad \texttt{if } n = 0 \texttt{ then } 1 \texttt{ else } 0 \\
\}
\end{array}
$$

This program can be encoded in λ-calculus as follows:

$$
\begin{array}{l}
\lambda n. \\
\quad (n\ (\lambda x.\ (\lambda x\ y.\ y))\ (\lambda x\ y.\ x))\quad \texttt{if isZero}(n)\ \texttt{then} \\
\qquad (\lambda f\ x.\ f\ x) \qquad\qquad\qquad\qquad\quad 1 \\
\qquad (\lambda f\ x.\ x)
\end{array}
$$

Detour revisited

```
int f (int n) {
  if n = 0 then 1 else 0
}
```

This program can be encoded in λ-calculus as follows:

$$
\begin{array}{ll}
\lambda n. & \\
\quad (n\ (\lambda x.\ (\lambda x\ y.\ y))\ (\lambda x\ y.\ x)) & \texttt{if isZero}(n)\ \texttt{then} \\
\quad\quad (\lambda f\ x.\ f\ x) & \quad 1 \\
\quad\quad (\lambda f\ x.\ x) & \texttt{else}\ 0
\end{array}
$$

Computation

Intuition: replace parameter by argument
this is called β-reduction

Example

$$(\lambda x\ y.\ f\ (y\ x))\ \ 5\ \ (\lambda x.\ x) \longrightarrow_\beta$$

Computation

Intuition: replace parameter by argument
this is called β-reduction

Example

$$(\lambda x\ y.\ f\ (y\ x))\ \ 5\ \ (\lambda x.\ x) \longrightarrow_\beta$$
$$(\lambda y.\ f\ (y\ 5))\ \ (\lambda x.\ x) \longrightarrow_\beta$$

Computation

Intuition: replace parameter by argument
this is called β-reduction

Example

$(\lambda x\ y.\ f\ (y\ x))\ \ 5\ \ (\lambda x.\ x) \longrightarrow_\beta$
$(\lambda y.\ f\ (y\ 5))\ \ (\lambda x.\ x) \longrightarrow_\beta$
$f\ ((\lambda x.\ x)\ 5) \longrightarrow_\beta$

Computation

Intuition: replace parameter by argument
this is called β-reduction

Example

$$(\lambda x\ y.\ f\ (y\ x))\ \ 5\ \ (\lambda x.\ x) \longrightarrow_\beta$$
$$(\lambda y.\ f\ (y\ 5))\ \ (\lambda x.\ x) \longrightarrow_\beta$$
$$f\ ((\lambda x.\ x)\ 5) \longrightarrow_\beta$$
$$f\ 5$$

Defining Computation

β reduction:

$$\frac{}{(\lambda x.\ s)\ t \longrightarrow_\beta s[x \leftarrow t]} \qquad \frac{s \longrightarrow_\beta s'}{(s\ t) \longrightarrow_\beta (s'\ t)}$$

$$\frac{t \longrightarrow_\beta t'}{(s\ t) \longrightarrow_\beta (s\ t')} \qquad \frac{s \longrightarrow_\beta s'}{\lambda x.s \longrightarrow_\beta \lambda x.s'}$$

Defining Computation

β reduction:

$$\frac{}{(\lambda x.\ s)\ t \longrightarrow_\beta s[x \leftarrow t]} \qquad \frac{s \longrightarrow_\beta s'}{(s\ t) \longrightarrow_\beta (s'\ t)}$$

$$\frac{t \longrightarrow_\beta t'}{(s\ t) \longrightarrow_\beta (s\ t')} \qquad \frac{s \longrightarrow_\beta s'}{\lambda x.s \longrightarrow_\beta \lambda x.s'}$$

Still to do: define $s[x \leftarrow t]$

Defining Substitution

Easy concept. Small problem: variable capture.

Example: $(\lambda x.\ x\ z)[z \leftarrow x]$

Defining Substitution

Easy concept. Small problem: variable capture.
Example: $(\lambda x.\ x\ z)[z \leftarrow x]$

We do **not** want: $(\lambda x.\ x\ x)$ as result.

What do we want?

Defining Substitution

Easy concept. Small problem: variable capture.
Example: $(\lambda x.\ x\ z)[z \leftarrow x]$

We do **not** want: $(\lambda x.\ x\ x)$ as result.

What do we want?

In $(\lambda y.\ y\ z)\ [z \leftarrow x] = (\lambda y.\ y\ x)$ there would be no problem.

So, solution is: rename bound variables.

Free Variables

Bound variables: in $(\lambda x.\ t)$, x is a bound variable.

Free Variables

Bound variables: in $(\lambda x.\ t)$, x is a bound variable.

Free variables FV of a term:

$$\begin{aligned} FV\ (x) &= \{x\} \\ FV\ (c) &= \{\} \\ FV\ (s\ t) &= FV(s) \cup FV(t) \\ FV\ (\lambda x.\ t) &= FV(t) \setminus \{x\} \end{aligned}$$

Example: $FV(\quad \lambda x.\ (\lambda y.\ (\lambda x.\ x)\ y)\ y\ x \quad)$

Free Variables

Bound variables: in $(\lambda x.\ t)$, x is a bound variable.

Free variables FV of a term:

$$\begin{aligned} FV\ (x) &= \{x\} \\ FV\ (c) &= \{\} \\ FV\ (s\ t) &= FV(s) \cup FV(t) \\ FV\ (\lambda x.\ t) &= FV(t) \setminus \{x\} \end{aligned}$$

Example: $FV(\quad \lambda x.\ (\lambda y.\ (\lambda x.\ x)\ y)\ y\ x \quad) = \{y\}$

Free Variables

Bound variables: in $(\lambda x.\ t)$, x is a bound variable.

Free variables FV of a term:

$$\begin{aligned} FV\ (x) &= \{x\} \\ FV\ (c) &= \{\} \\ FV\ (s\ t) &= FV(s) \cup FV(t) \\ FV\ (\lambda x.\ t) &= FV(t) \setminus \{x\} \end{aligned}$$

Example: $FV(\quad \lambda x.\ (\lambda y.\ (\lambda x.\ x)\ y)\ y\ x \quad) = \{y\}$

Term t is called **closed** if $FV(t) = \{\}$

Free Variables

Bound variables: in $(\lambda x.\ t)$, x is a bound variable.

Free variables FV of a term:

$$\begin{aligned} FV\ (x) &= \{x\} \\ FV\ (c) &= \{\} \\ FV\ (s\ t) &= FV(s) \cup FV(t) \\ FV\ (\lambda x.\ t) &= FV(t) \setminus \{x\} \end{aligned}$$

Example: $FV(\quad \lambda x.\ (\lambda y.\ (\lambda x.\ x)\ y)\ y\ x \quad) = \{y\}$

Term t is called **closed** if $FV(t) = \{\}$

The substitution example, $(\lambda x.\ x\ z)[z \leftarrow x]$, is problematic because the bound variable x is a free variable of the replacement term "x".

Substitution

$$
\begin{aligned}
x\ [x \leftarrow t] &= t \\
y\ [x \leftarrow t] &= y \qquad \text{if } x \neq y \\
c\ [x \leftarrow t] &= c
\end{aligned}
$$

$$
(s_1\ s_2)\ [x \leftarrow t] =
$$

Substitution

$$
\begin{array}{lll}
x\ [x \leftarrow t] & = t & \\
y\ [x \leftarrow t] & = y & \text{if } x \neq y \\
c\ [x \leftarrow t] & = c &
\end{array}
$$

$$(s_1\ s_2)\ [x \leftarrow t] = (s_1[x \leftarrow t]\ s_2[x \leftarrow t])$$

$$(\lambda x.\ s)\ [x \leftarrow t] =$$

Substitution

$$
\begin{aligned}
x\ [x \leftarrow t] &= t \\
y\ [x \leftarrow t] &= y \qquad \text{if } x \neq y \\
c\ [x \leftarrow t] &= c
\end{aligned}
$$

$$
(s_1\ s_2)\ [x \leftarrow t] = (s_1[x \leftarrow t]\ s_2[x \leftarrow t])
$$

$$
\begin{aligned}
(\lambda x.\ s)\ [x \leftarrow t] &= (\lambda x.\ s) \\
(\lambda y.\ s)\ [x \leftarrow t] &=
\end{aligned}
$$

Substitution

$$
\begin{array}{lll}
x\ [x \leftarrow t] & = t & \\
y\ [x \leftarrow t] & = y & \text{if } x \neq y \\
c\ [x \leftarrow t] & = c & \\
 & & \\
(s_1\ s_2)\ [x \leftarrow t] & = (s_1[x \leftarrow t]\ s_2[x \leftarrow t]) & \\
 & & \\
(\lambda x.\ s)\ [x \leftarrow t] & = (\lambda x.\ s) & \\
(\lambda y.\ s)\ [x \leftarrow t] & = (\lambda y.\ s[x \leftarrow t]) & \text{if } x \neq y \text{ and } y \notin FV(t) \\
(\lambda y.\ s)\ [x \leftarrow t] & = &
\end{array}
$$

Substitution

$$
\begin{array}{lll}
x\ [x \leftarrow t] & = t & \\
y\ [x \leftarrow t] & = y & \text{if } x \neq y \\
c\ [x \leftarrow t] & = c & \\
 & & \\
(s_1\ s_2)\ [x \leftarrow t] & = (s_1[x \leftarrow t]\ s_2[x \leftarrow t]) & \\
 & & \\
(\lambda x.\ s)\ [x \leftarrow t] & = (\lambda x.\ s) & \\
(\lambda y.\ s)\ [x \leftarrow t] & = (\lambda y.\ s[x \leftarrow t]) & \text{if } x \neq y \text{ and } y \notin FV(t) \\
(\lambda y.\ s)\ [x \leftarrow t] & = (\lambda z.\ s[y \leftarrow z][x \leftarrow t]) & \text{if } x \neq y \\
 & & \text{and } z \notin FV(t) \cup FV(s)
\end{array}
$$

Substitution Example

$$(x \;\; (\lambda x.\; x) \;\; (\lambda y.\; z\; x))[x \leftarrow y]$$

Substitution Example

$$
\begin{array}{rl}
 & (x\ \ (\lambda x.\ x)\ \ (\lambda y.\ z\ x))[x \leftarrow y] \\
= & (x[x \leftarrow y])\ \ ((\lambda x.\ x)[x \leftarrow y])\ \ ((\lambda y.\ z\ x)[x \leftarrow y])
\end{array}
$$

Substitution Example

$$
\begin{array}{rl}
 & (x\ \ (\lambda x.\ x)\ \ (\lambda y.\ z\ x))[x \leftarrow y] \\
= & (x[x \leftarrow y])\ \ ((\lambda x.\ x)[x \leftarrow y])\ \ ((\lambda y.\ z\ x)[x \leftarrow y]) \\
= & y\ (\lambda x.\ x)\ (\lambda y'.\ z\ y)
\end{array}
$$

α Conversion

Bound names are irrelevant:

$\lambda x.\ x$ and $\lambda y.\ y$ denote the same function.

α conversion:

$s =_\alpha t$ means $s = t$ up to renaming of bound variables.

α Conversion

Bound names are irrelevant:

$\lambda x.\ x$ and $\lambda y.\ y$ denote the same function.

α conversion:

$s =_\alpha t$ means $s = t$ up to renaming of bound variables.

Formally:

$$\frac{y \notin FV(t)}{(\lambda x.\ t) \longrightarrow_\alpha (\lambda y.\ t[x \leftarrow y])} \qquad \frac{s \longrightarrow_\alpha s'}{(s\ t) \longrightarrow_\alpha (s'\ t)}$$

$$\frac{t \longrightarrow_\alpha t'}{(s\ t) \longrightarrow_\alpha (s\ t')} \qquad \frac{s \longrightarrow_\alpha s'}{(\lambda x.\ s) \longrightarrow_\alpha (\lambda x.\ s')}$$

α Conversion

Bound names are irrelevant:

$\lambda x.\ x$ and $\lambda y.\ y$ denote the same function.

α conversion:

$s =_\alpha t$ means $s = t$ up to renaming of bound variables.

Formally:

$$\frac{y \notin FV(t)}{(\lambda x.\ t) \longrightarrow_\alpha (\lambda y.\ t[x \leftarrow y])} \qquad \frac{s \longrightarrow_\alpha s'}{(s\ t) \longrightarrow_\alpha (s'\ t)}$$

$$\frac{t \longrightarrow_\alpha t'}{(s\ t) \longrightarrow_\alpha (s\ t')} \qquad \frac{s \longrightarrow_\alpha s'}{(\lambda x.\ s) \longrightarrow_\alpha (\lambda x.\ s')}$$

$$s =_\alpha t \quad \text{iff} \quad s \longrightarrow_\alpha^* t$$

($\longrightarrow_\alpha^*$ = transitive, reflexive closure of $\longrightarrow_\alpha$ = multiple steps)

α Conversion

Examples:

$$x\ (\lambda x\ y.\ x\ y)$$

α Conversion

Examples:

$$\begin{array}{rl} & x\ (\lambda x\ y.\ x\ y) \\ =_\alpha & x\ (\lambda y\ x.\ y\ x) \end{array}$$

α Conversion

Examples:

$$
\begin{array}{rl}
 & x\ (\lambda x\ y.\ x\ y) \\
=_\alpha & x\ (\lambda y\ x.\ y\ x) \\
=_\alpha & x\ (\lambda z\ y.\ z\ y)
\end{array}
$$

α Conversion

Examples:

$$
\begin{array}{ll}
 & x\ (\lambda x\ y.\ x\ y) \\
=_\alpha & x\ (\lambda y\ x.\ y\ x) \\
=_\alpha & x\ (\lambda z\ y.\ z\ y) \\
\neq_\alpha & z\ (\lambda z\ y.\ z\ y)
\end{array}
$$

α Conversion

Examples:

$$
\begin{array}{ll}
 & x\ (\lambda x\ y.\ x\ y) \\
=_\alpha & x\ (\lambda y\ x.\ y\ x) \\
=_\alpha & x\ (\lambda z\ y.\ z\ y) \\
\neq_\alpha & z\ (\lambda z\ y.\ z\ y) \\
\neq_\alpha & z\ (\lambda x\ x.\ x\ x)
\end{array}
$$

Back to β

We have defined β reduction: $\longrightarrow_\beta$
Some notation and concepts:

- β **conversion:** $s =_\beta t$ iff $\exists n.\ s \longrightarrow_\beta^* n \wedge t \longrightarrow_\beta^* n$

Back to β

We have defined β reduction: $\longrightarrow_\beta$
Some notation and concepts:

- β **conversion:** $s =_\beta t$ iff $\exists n.\ s \longrightarrow_\beta^* n \wedge t \longrightarrow_\beta^* n$
- t is **reducible** if there is an s such that $t \longrightarrow_\beta s$

Back to β

We have defined β reduction: $\longrightarrow_\beta$
Some notation and concepts:

- β **conversion:** $s =_\beta t$ iff $\exists n.\ s \longrightarrow_\beta^* n \wedge t \longrightarrow_\beta^* n$
- t is **reducible** if there is an s such that $t \longrightarrow_\beta s$
- $(\lambda x.\ s)\ t$ is called a **redex** (reducible expression)

Back to β

We have defined β reduction: $\longrightarrow_\beta$
Some notation and concepts:

- β **conversion:** $s =_\beta t$ iff $\exists n.\ s \longrightarrow_\beta^* n \wedge t \longrightarrow_\beta^* n$
- t is **reducible** if there is an s such that $t \longrightarrow_\beta s$
- $(\lambda x.\ s)\ t$ is called a **redex** (reducible expression)
- t is reducible iff it contains a redex

Back to β

We have defined β reduction: $\longrightarrow_\beta$
Some notation and concepts:

- β **conversion:** $s =_\beta t$ iff $\exists n.\ s \longrightarrow_\beta^* n \wedge t \longrightarrow_\beta^* n$
- t is **reducible** if there is an s such that $t \longrightarrow_\beta s$
- $(\lambda x.\ s)\ t$ is called a **redex** (reducible expression)
- t is reducible iff it contains a redex
- if it is not reducible, t is in **normal form**

Does every λ term have a normal form?

Example:

$$(\lambda x.\ x\ x)\ (\lambda x.\ x\ x)\ \longrightarrow_\beta$$

Does every λ term have a normal form?

Example:

$$(\lambda x.\ x\ x)\ (\lambda x.\ x\ x) \longrightarrow_\beta$$
$$(\lambda x.\ x\ x)\ (\lambda x.\ x\ x) \longrightarrow_\beta$$

Does every λ term have a normal form?

No!

Example:

$$
\begin{aligned}
(\lambda x.\ x\ x)\ (\lambda x.\ x\ x) &\longrightarrow_\beta \\
(\lambda x.\ x\ x)\ (\lambda x.\ x\ x) &\longrightarrow_\beta \\
(\lambda x.\ x\ x)\ (\lambda x.\ x\ x) &\longrightarrow_\beta \ldots
\end{aligned}
$$

Does every λ term have a normal form?

No!

Example:

$$
\begin{aligned}
(\lambda x.\ x\ x)\ (\lambda x.\ x\ x) &\longrightarrow_\beta \\
(\lambda x.\ x\ x)\ (\lambda x.\ x\ x) &\longrightarrow_\beta \\
(\lambda x.\ x\ x)\ (\lambda x.\ x\ x) &\longrightarrow_\beta \ldots
\end{aligned}
$$

(but: $(\lambda x\ y.\ y)\ ((\lambda x.\ x\ x)\ (\lambda x.\ x\ x)) \longrightarrow_\beta \lambda y.\ y$)

Does every λ term have a normal form?

No!

Example:

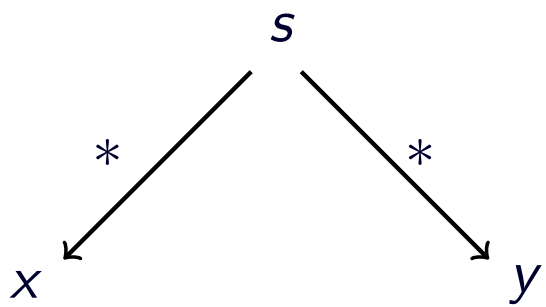
$$
\begin{aligned}
(\lambda x.\ x\ x)\ (\lambda x.\ x\ x) &\longrightarrow_\beta \\
(\lambda x.\ x\ x)\ (\lambda x.\ x\ x) &\longrightarrow_\beta \\
(\lambda x.\ x\ x)\ (\lambda x.\ x\ x) &\longrightarrow_\beta \ldots
\end{aligned}
$$

(but: $(\lambda x\ y.\ y)\ ((\lambda x.\ x\ x)\ (\lambda x.\ x\ x)) \longrightarrow_\beta \lambda y.\ y$)

λ calculus is not terminating

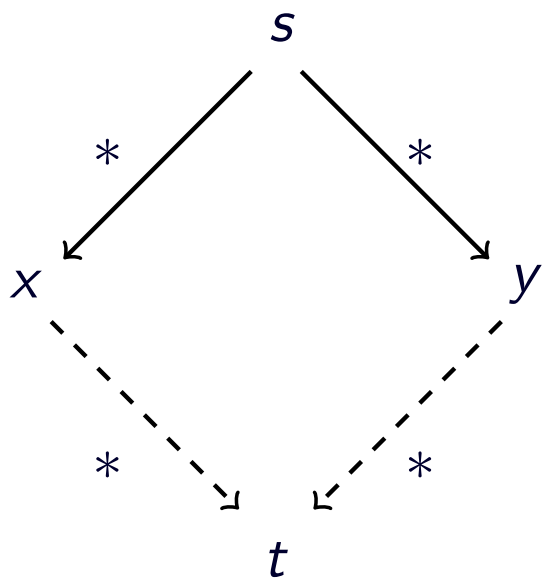
β reduction is confluent

Confluence: $s \longrightarrow_\beta^* x \wedge s \longrightarrow_\beta^* y \Longrightarrow \exists t.\ x \longrightarrow_\beta^* t \wedge y \longrightarrow_\beta^* t$



β reduction is confluent

Confluence: $s \longrightarrow_\beta^* x \wedge s \longrightarrow_\beta^* y \Longrightarrow \exists t.\ x \longrightarrow_\beta^* t \wedge y \longrightarrow_\beta^* t$



Order of reduction does not matter for result
Normal forms in λ calculus are unique

β reduction is confluent

Example:

$(\lambda x\ y.\ y)\ ((\lambda x.\ x\ x)\ a)$

$(\lambda x\ y.\ y)\ ((\lambda x.\ x\ x)\ a)$

β reduction is confluent

Example:

$(\lambda x\ y.\ y)\ ((\lambda x.\ x\ x)\ a) \longrightarrow_\beta (\lambda x\ y.\ y)\ (a\ a)$

$(\lambda x\ y.\ y)\ ((\lambda x.\ x\ x)\ a) \longrightarrow_\beta \lambda y.\ y$

β reduction is confluent

Example:

$(\lambda x\ y.\ y)\ ((\lambda x.\ x\ x)\ a) \longrightarrow_\beta (\lambda x\ y.\ y)\ (a\ a) \longrightarrow_\beta \lambda y.\ y$

$(\lambda x\ y.\ y)\ ((\lambda x.\ x\ x)\ a) \longrightarrow_\beta \lambda y.\ y$

So, what can you do with λ calculus?

λ calculus is very expressive, you can encode:

- logic, set theory
- turing machines, functional programs, etc.

Examples:

So, what can you do with λ calculus?

λ calculus is very expressive, you can encode:

- logic, set theory
- turing machines, functional programs, etc.

Examples:

$$\texttt{true} \equiv \lambda x\ y.\ x$$
$$\texttt{false} \equiv \lambda x\ y.\ y$$
$$\texttt{if} \equiv \lambda z\ x\ y.\ z\ x\ y$$

So, what can you do with λ calculus?

λ calculus is very expressive, you can encode:

- logic, set theory
- turing machines, functional programs, etc.

Examples:

$$\texttt{true} \equiv \lambda x\ y.\ x \qquad \texttt{if true } x\ y \longrightarrow_\beta^* x$$

$$\texttt{false} \equiv \lambda x\ y.\ y \qquad \texttt{if false } x\ y \longrightarrow_\beta^* y$$

$$\texttt{if} \equiv \lambda z\ x\ y.\ z\ x\ y$$

So, what can you do with λ calculus?

λ calculus is very expressive, you can encode:

- logic, set theory
- turing machines, functional programs, etc.

Examples:

$$\texttt{true} \equiv \lambda x\ y.\ x \qquad \texttt{if true } x\ y \longrightarrow_\beta^* x$$
$$\texttt{false} \equiv \lambda x\ y.\ y \qquad \texttt{if false } x\ y \longrightarrow_\beta^* y$$
$$\texttt{if} \equiv \lambda z\ x\ y.\ z\ x\ y$$

Now, `not`, `and`, `or`, etc is easy:

So, what can you do with λ calculus?

λ calculus is very expressive, you can encode:

- logic, set theory
- turing machines, functional programs, etc.

Examples:

$$\texttt{true} \equiv \lambda x\ y.\ x \qquad \texttt{if true } x\ y \longrightarrow_\beta^* x$$
$$\texttt{false} \equiv \lambda x\ y.\ y \qquad \texttt{if false } x\ y \longrightarrow_\beta^* y$$
$$\texttt{if} \equiv \lambda z\ x\ y.\ z\ x\ y$$

Now, `not`, `and`, `or`, etc is easy:

$$\texttt{not} \equiv \lambda x.\ \texttt{if } x \texttt{ false true}$$
$$\texttt{and} \equiv \lambda x\ y.\ \texttt{if } x\ y \texttt{ false}$$
$$\texttt{or} \equiv \lambda x\ y.\ \texttt{if } x \texttt{ true } y$$

Fix Points

$$(\lambda x\ f.\ f\ (x\ x\ f))\ \ (\lambda x\ f.\ f\ (x\ x\ f))\ \ t \longrightarrow_\beta$$

Fix Points

$$(\lambda x\ f.\ f\ (x\ x\ f))\ \ (\lambda x\ f.\ f\ (x\ x\ f))\ \ t \longrightarrow_\beta$$
$$(\lambda f.\ f\ ((\lambda x\ f.\ f\ (x\ x\ f))\ \ (\lambda x\ f.\ f\ (x\ x\ f))\ f))\ \ t \longrightarrow_\beta$$

Fix Points

$$(\lambda x\ f.\ f\ (x\ x\ f))\ \ (\lambda x\ f.\ f\ (x\ x\ f))\ \ t \longrightarrow_\beta$$
$$(\lambda f.\ f\ ((\lambda x\ f.\ f\ (x\ x\ f))\ \ (\lambda x\ f.\ f\ (x\ x\ f))\ f))\ \ t \longrightarrow_\beta$$
$$t\ \ ((\lambda x\ f.\ f\ (x\ x\ f))\ (\lambda x\ f.\ f\ (x\ x\ f))\ t)$$

Fix Points

$(\lambda x\ f.\ f\ (x\ x\ f))\ \ (\lambda x\ f.\ f\ (x\ x\ f))\ \ t \longrightarrow_\beta$
$(\lambda f.\ f\ ((\lambda x\ f.\ f\ (x\ x\ f))\ \ (\lambda x\ f.\ f\ (x\ x\ f))\ f))\ \ t \longrightarrow_\beta$
$t\ \ ((\lambda x\ f.\ f\ (x\ x\ f))\ (\lambda x\ f.\ f\ (x\ x\ f))\ t)$

$\mu = (\lambda x f.\ f\ (x\ x\ f))\ (\lambda x f.\ f\ (x\ x\ f))$
$\mu\ t \longrightarrow_\beta t\ (\mu\ t) \longrightarrow_\beta t\ (t\ (\mu\ t)) \longrightarrow_\beta t\ (t\ (t\ (\mu\ t))) \longrightarrow_\beta \ldots$

Fix Points

$(\lambda x\ f.\ f\ (x\ x\ f))\ \ (\lambda x\ f.\ f\ (x\ x\ f))\ \ t \longrightarrow_\beta$
$(\lambda f.\ f\ ((\lambda x\ f.\ f\ (x\ x\ f))\ \ (\lambda x\ f.\ f\ (x\ x\ f))\ f))\ \ t \longrightarrow_\beta$
$t\ \ ((\lambda x\ f.\ f\ (x\ x\ f))\ (\lambda x\ f.\ f\ (x\ x\ f))\ t)$

$\mu = (\lambda x f.\ f\ (x\ x\ f))\ (\lambda x f.\ f\ (x\ x\ f))$
$\mu\ t \longrightarrow_\beta t\ (\mu\ t) \longrightarrow_\beta t\ (t\ (\mu\ t)) \longrightarrow_\beta t\ (t\ (t\ (\mu\ t))) \longrightarrow_\beta \ldots$

$(\lambda x f.\ f\ (x\ x\ f))\ (\lambda x f.\ f\ (x\ x\ f))$ is Turing's fix point operator

Nice, but ...

As a mathematical foundation, (untyped) λ does not work. **It is inconsistent.**

Nice, but ...

As a mathematical foundation, (untyped) λ does not work. **It is inconsistent.**

- **Russell** (1901): Paradox $R \equiv \{X | X \notin X\}$
- **Church** (1930): λ calculus as logic, `true`, `false`, $\wedge$, ... as λ terms

Russell's paradox in λ-calculus:

Nice, but ...

As a mathematical foundation, (untyped) λ does not work. **It is inconsistent.**

- **Russell** (1901): Paradox $R \equiv \{X | X \notin X\}$
- **Church** (1930): λ calculus as logic, `true`, `false`, $\wedge$, ... as λ terms

Russell's paradox in λ-calculus:

you can write $R \equiv \lambda x.\ \texttt{not}\ (x\ x)$

Nice, but ...

As a mathematical foundation, (untyped) λ does not work. **It is inconsistent.**

- **Russell** (1901): Paradox $R \equiv \{X | X \notin X\}$
- **Church** (1930): λ calculus as logic, `true`, `false`, $\wedge$, ... as λ terms

Russell's paradox in λ-calculus:

you can write $R \equiv \lambda x.\ \texttt{not}\ (x\ x)$

and get $(R\ R) =_\beta \texttt{not}\ (R\ R)$

Nice, but ...

As a mathematical foundation, (untyped) λ does not work. **It is inconsistent.**

- **Russell** (1901): Paradox $R \equiv \{X | X \notin X\}$
- **Church** (1930): λ calculus as logic, `true`, `false`, $\wedge$, ... as λ terms

Russell's paradox in λ-calculus:

you can write	$R \equiv \lambda x.\ \texttt{not}\ (x\ x)$
and get	$(R\ R) =_\beta \texttt{not}\ (R\ R)$
because	$(R\ R) = (\lambda x.\ \texttt{not}\ (x\ x))\ R \longrightarrow_\beta \texttt{not}\ (R\ R)$

Summary of what we learned so far

- λ calculus syntax
- free variables, substitution
- α conversion
- β reduction
- β reduction is confluent
- λ calculus is very expressive (Turing complete)
- λ calculus is inconsistent (as a logic)

Exercise

- Reduce $(\lambda x.\ y\ (\lambda v.\ x\ v))\ (\lambda y.\ v\ y)$ to β normal form.

λ calculus is inconsistent

Can find term R such that $R\ R\ =_\beta\ \mathtt{not}(R\ R)$

λ calculus is inconsistent

Can find term R such that $R\ R\ =_\beta\ \mathtt{not}(R\ R)$

Solution: rule out ill-formed terms by using types.
(Church 1940)

Introducing types

Idea: assign a type to each "sensible" λ term.

Examples:

Introducing types

Idea: assign a type to each "sensible" λ term.

Examples:

- for *term* t *has type* α write $t :: \alpha$

Introducing types

Idea: assign a type to each "sensible" λ term.

Examples:

- for *term* t *has type* α write $t :: \alpha$
- if x has type α then $\lambda x.\ x$ is a function from α to α
  Write: $(\lambda x.\ x) :: \alpha \Rightarrow \alpha$

Introducing types

Idea: assign a type to each "sensible" λ term.

Examples:

- for *term* t *has type* α write $t :: \alpha$
- if x has type α then $\lambda x.\ x$ is a function from α to α
  Write: $(\lambda x.\ x) :: \alpha \Rightarrow \alpha$
- for $s\ t$ to be sensible:
  s must be a function
  t must be right type for parameter

  If $s :: \alpha \Rightarrow \beta$ and $t :: \alpha$ then $(s\ t) :: \beta$

That's it!

That's it!

Now Formally

Syntax for $\lambda^{\to}$

Terms: $t \quad ::= \quad v \mid c \mid (t\ t) \mid (\lambda x.\ t)$

$v, x \in V, \quad c \in C, \quad V, C$ sets of names

Types: $\tau \quad ::= \quad \mathtt{b} \mid \nu \mid \tau \Rightarrow \tau$

$\mathtt{b} \in \{\mathtt{bool}, \mathtt{int}, \ldots\}$ base types

$\nu \in \{\alpha, \beta, \ldots\}$ type variables

$$\alpha \Rightarrow \beta \Rightarrow \gamma \quad = \quad \alpha \Rightarrow (\beta \Rightarrow \gamma)$$

Syntax for $\lambda^{\to}$

Terms: $t \quad ::= \quad v \mid c \mid (t\ t) \mid (\lambda x.\ t)$

$v, x \in V, \quad c \in C, \quad V, C$ sets of names

Types: $\tau \quad ::= \quad \mathtt{b} \mid \nu \mid \tau \Rightarrow \tau$

$\mathtt{b} \in \{\mathtt{bool}, \mathtt{int}, \ldots\}$ base types

$\nu \in \{\alpha, \beta, \ldots\}$ type variables

$$\alpha \Rightarrow \beta \Rightarrow \gamma \quad = \quad \alpha \Rightarrow (\beta \Rightarrow \gamma)$$

Context Γ:

Γ: function from variable and constant names to types.

Syntax for $\lambda^{\to}$

Terms: $t \;::=\; v \mid c \mid (t\ t) \mid (\lambda x.\ t)$
$v, x \in V, \quad c \in C, \quad V, C$ sets of names

Types: $\tau \;::=\; \mathtt{b} \mid \nu \mid \tau \Rightarrow \tau$
$\mathtt{b} \in \{\mathtt{bool}, \mathtt{int}, \ldots\}$ base types
$\nu \in \{\alpha, \beta, \ldots\}$ type variables

$$\alpha \Rightarrow \beta \Rightarrow \gamma \quad = \quad \alpha \Rightarrow (\beta \Rightarrow \gamma)$$

Context Γ:
Γ: function from variable and constant names to types.

Term t has type τ in context Γ: $\Gamma \vdash t :: \tau$

Examples

$\Gamma \vdash (\lambda x.\ x) :: \alpha \Rightarrow \alpha$

Examples

$\Gamma \vdash (\lambda x.\ x) :: \alpha \Rightarrow \alpha$

$[y \leftarrow \texttt{int}] \vdash y :: \texttt{int}$

Examples

$\Gamma \vdash (\lambda x.\ x) :: \alpha \Rightarrow \alpha$

$[y \leftarrow \texttt{int}] \vdash y :: \texttt{int}$

$[z \leftarrow \texttt{bool}] \vdash (\lambda y.\ y)\ z :: \texttt{bool}$

Examples

$\Gamma \vdash (\lambda x.\ x) :: \alpha \Rightarrow \alpha$

$[y \leftarrow \texttt{int}] \vdash y :: \texttt{int}$

$[z \leftarrow \texttt{bool}] \vdash (\lambda y.\ y)\ z :: \texttt{bool}$

$[] \vdash \lambda f\ x.\ f\ x :: (\alpha \Rightarrow \beta) \Rightarrow \alpha \Rightarrow \beta$

Examples

$\Gamma \vdash (\lambda x.\ x) :: \alpha \Rightarrow \alpha$

$[y \leftarrow \texttt{int}] \vdash y :: \texttt{int}$

$[z \leftarrow \texttt{bool}] \vdash (\lambda y.\ y)\ z :: \texttt{bool}$

$[] \vdash \lambda f\ x.\ f\ x :: (\alpha \Rightarrow \beta) \Rightarrow \alpha \Rightarrow \beta$

A term t is **well typed** or **type correct**
if there are Γ and τ such that $\Gamma \vdash t :: \tau$

Type Checking Rules

Variables: $$\frac{}{\Gamma \vdash x :: \Gamma(x)}$$

Type Checking Rules

Variables:

$$\frac{}{\Gamma \vdash x :: \Gamma(x)}$$

Application:

$$\frac{}{\Gamma \vdash (t_1\ t_2) :: \tau}$$

Type Checking Rules

Variables:

$$\frac{}{\Gamma \vdash x :: \Gamma(x)}$$

Application:

$$\frac{\Gamma \vdash t_1 :: \tau_2 \Rightarrow \tau \quad \Gamma \vdash t_2 :: \tau_2}{\Gamma \vdash (t_1\ t_2) :: \tau}$$

Type Checking Rules

Variables:

$$\frac{}{\Gamma \vdash x :: \Gamma(x)}$$

Application:

$$\frac{\Gamma \vdash t_1 :: \tau_2 \Rightarrow \tau \quad \Gamma \vdash t_2 :: \tau_2}{\Gamma \vdash (t_1\ t_2) :: \tau}$$

Abstraction:

$$\frac{}{\Gamma \vdash (\lambda x.\ t) :: \tau_x \Rightarrow \tau}$$

Type Checking Rules

Variables:

$$\frac{}{\Gamma \vdash x :: \Gamma(x)}$$

Application:

$$\frac{\Gamma \vdash t_1 :: \tau_2 \Rightarrow \tau \quad \Gamma \vdash t_2 :: \tau_2}{\Gamma \vdash (t_1\ t_2) :: \tau}$$

Abstraction:

$$\frac{\Gamma[x \leftarrow \tau_x] \vdash t :: \tau}{\Gamma \vdash (\lambda x.\ t) :: \tau_x \Rightarrow \tau}$$

What about β reduction?

What about β reduction?

Definition of β reduction stays the same.

What about β reduction?

Definition of β reduction stays the same.

Fact: Well typed terms stay well typed after β reduction

Formally: $\Gamma \vdash s :: \tau \ \wedge \ s \longrightarrow_\beta t \Longrightarrow \Gamma \vdash t :: \tau$

What about β reduction?

Definition of β reduction stays the same.

Fact: Well typed terms stay well typed after β reduction

Formally: $\Gamma \vdash s :: \tau \ \wedge \ s \longrightarrow_\beta t \Longrightarrow \Gamma \vdash t :: \tau$

This property is called **subject reduction**

Exercise

Derive the normal form by β-reducing the following term:

$$(\lambda f\ x.\ f\ x)(\lambda y.\ y)\ z$$

What about termination?

What about termination?

β reduction in $\lambda^{\rightarrow}$ always terminates.

(Alan Turing, 1942)

What about termination?

β reduction in $\lambda^{\to}$ always terminates.

(Alan Turing, 1942)

- **$=_\beta$ is decidable**
  To decide if $s =_\beta t$, reduce s and t to normal form (always exists, because $\longrightarrow_\beta$ terminates), and compare result.

What about termination?

β reduction in $\lambda^{\to}$ always terminates.

(Alan Turing, 1942)

- **$=_\beta$ is decidable**
  To decide if $s =_\beta t$, reduce s and t to normal form (always exists, because $\longrightarrow_\beta$ terminates), and compare result.
- **$=_{\alpha\beta}$ is decidable**

We have learned so far...

- Simply typed lambda calculus: $\lambda^{\rightarrow}$
- Typing rules for $\lambda^{\rightarrow}$, type variables, type contexts
- β-reduction in $\lambda^{\rightarrow}$ satisfies subject reduction
- β-reduction in $\lambda^{\rightarrow}$ always terminates

Defining Higher Order Logic

What is Higher Order Logic?

- **Propositional Logic:**
  - no quantifiers
  - all variables have type bool

What is Higher Order Logic?

- **Propositional Logic:**
  - no quantifiers
  - all variables have type bool
- **First Order Logic:**
  - quantification over values, but not over functions and predicates,
  - terms and formulas syntactically distinct

What is Higher Order Logic?

- **Propositional Logic:**
  - no quantifiers
  - all variables have type bool
- **First Order Logic:**
  - quantification over values, but not over functions and predicates,
  - terms and formulas syntactically distinct
- **Higher Order Logic:**
  - quantification over everything, including predicates
  - consistency by types
  - formula = term of type bool
  - definition built on $\lambda^{\rightarrow}$ with certain default types and constants

Defining Higher Order Logic

Default types:

Defining Higher Order Logic

Default types:

bool

Defining Higher Order Logic

Default types:

$$\text{bool} \qquad \_ \Rightarrow \_$$

Defining Higher Order Logic

Default types:

bool $\quad\quad$ _ $\Rightarrow$ _ $\quad\quad$ ind

Defining Higher Order Logic

Default types:

bool $\quad$ _ $\Rightarrow$ _ $\quad$ ind

- **bool**
- $\Rightarrow$ sometimes called *fun*

Defining Higher Order Logic

Default types:

bool $\quad$ _ $\Rightarrow$ _ $\quad$ ind

- **bool**
- $\Rightarrow$ sometimes called *fun*

Default Constants:

Defining Higher Order Logic

Default types:

bool _ ⇒ _ ind

- **bool**
- ⇒ sometimes called *fun*

Default Constants:

$$= \quad :: \quad \alpha \Rightarrow \alpha \Rightarrow \mathit{bool}$$

Higher Order Abstract Syntax

Problem: Define syntax for binders like $\forall$ and $\exists$

Higher Order Abstract Syntax

Problem: Define syntax for binders like $\forall$ and $\exists$

One approach: $\forall :: \mathit{var} \Rightarrow \mathit{term} \Rightarrow \mathit{bool}$
Drawback: need to think about substitution, α conversion again.

Higher Order Abstract Syntax

Problem: Define syntax for binders like $\forall$ and $\exists$

One approach: $\forall :: \mathit{var} \Rightarrow \mathit{term} \Rightarrow \mathit{bool}$
Drawback: need to think about substitution, α conversion again.

But: Already have binder, substitution, α conversion in host language

$$\lambda$$

Higher Order Abstract Syntax

Problem: Define syntax for binders like $\forall$ and $\exists$

One approach: $\forall :: \mathit{var} \Rightarrow \mathit{term} \Rightarrow \mathit{bool}$
Drawback: need to think about substitution, α conversion again.

But: Already have binder, substitution, α conversion in host language

$$\lambda$$

So: Use λ to encode all other binders.

Higher Order Abstract Syntax

Example:

$$\text{ALL} :: (\alpha \Rightarrow \mathit{bool}) \Rightarrow \mathit{bool}$$

HOAS **usual syntax**

Higher Order Abstract Syntax

Example:

$$\text{ALL} :: (\alpha \Rightarrow \mathit{bool}) \Rightarrow \mathit{bool}$$

HOAS	**usual syntax**
ALL $(\lambda x.\ x = 2)$	

Higher Order Abstract Syntax

Example:

$$\text{ALL} :: (\alpha \Rightarrow \mathit{bool}) \Rightarrow \mathit{bool}$$

HOAS	**usual syntax**
$\text{ALL}\ (\lambda x.\ x = 2)$	$\forall x.\ x = 2$

Higher Order Abstract Syntax

Example:

$$\text{ALL} :: (\alpha \Rightarrow \mathit{bool}) \Rightarrow \mathit{bool}$$

HOAS	**usual syntax**
ALL $(\lambda x.\ x = 2)$	$\forall x.\ x = 2$
ALL P	

Higher Order Abstract Syntax

Example:

$$\text{ALL} :: (\alpha \Rightarrow \mathit{bool}) \Rightarrow \mathit{bool}$$

HOAS	**usual syntax**
ALL $(\lambda x.\ x = 2)$	$\forall x.\ x = 2$
ALL P	$\forall x.\ P\ x$

Back to HOL

Base: $bool, \Rightarrow, ind \quad =$

And the rest is

Back to HOL

Base: $bool, \Rightarrow, ind \quad =$

And the rest is definitions:

$\mathsf{True} \quad \equiv$

$P \wedge Q \quad \equiv$

$P \longrightarrow Q \quad \equiv$

$\mathsf{All}\ P \quad \equiv$

$\mathsf{Ex}\ P \quad \equiv$

$\mathsf{False} \quad \equiv$

$\neg P \quad \equiv$

$P \vee Q \quad \equiv$

Back to HOL

Base: $bool, \Rightarrow, ind \quad =$

And the rest is definitions:

$$
\begin{array}{lcl}
\text{True} & \equiv & (\lambda x.\ x) = (\lambda x.\ x) \\
P \wedge Q & \equiv & \\
P \longrightarrow Q & \equiv & \\
\text{All } P & \equiv & \\
\text{Ex } P & \equiv & \\
\text{False} & \equiv & \\
\neg P & \equiv & \\
P \vee Q & \equiv &
\end{array}
$$

Back to HOL

Base: $bool, \Rightarrow, ind \quad =$

And the rest is definitions:

$$
\begin{array}{lcl}
\text{True} & \equiv & (\lambda x.\ x) = (\lambda x.\ x) \\
P \wedge Q & \equiv & \lambda p\ q.\ ((\lambda f.\ f\ p\ q) = (\lambda f.\ f\ \text{True}\ \text{True})) \\
P \longrightarrow Q & \equiv & \\
\text{All}\ P & \equiv & \\
\text{Ex}\ P & \equiv & \\
\text{False} & \equiv & \\
\neg P & \equiv & \\
P \vee Q & \equiv &
\end{array}
$$

Back to HOL

Base: $bool, \Rightarrow, ind \quad =$

And the rest is definitions:

$$
\begin{array}{lcl}
\text{True} & \equiv & (\lambda x.\ x) = (\lambda x.\ x) \\
P \wedge Q & \equiv & \lambda p\ q.\ ((\lambda f.\ f\ p\ q) = (\lambda f.\ f\ \text{True}\ \text{True})) \\
P \longrightarrow Q & \equiv & \lambda p\ q.\ ((p \wedge q) = p) \\
\text{All}\ P & \equiv & \\
\text{Ex}\ P & \equiv & \\
\text{False} & \equiv & \\
\neg P & \equiv & \\
P \vee Q & \equiv &
\end{array}
$$

Back to HOL

Base: $bool, \Rightarrow, ind \quad =$

And the rest is definitions:

$$
\begin{array}{lcl}
\text{True} & \equiv & (\lambda x.\ x) = (\lambda x.\ x) \\
P \wedge Q & \equiv & \lambda p\ q.\ ((\lambda f.\ f\ p\ q) = (\lambda f.\ f\ \text{True}\ \text{True})) \\
P \longrightarrow Q & \equiv & \lambda p\ q.\ ((p \wedge q) = p) \\
\text{All}\ P & \equiv & P = (\lambda x.\ \text{True}) \\
\text{Ex}\ P & \equiv & \forall Q.\ (\forall x.\ P\ x \longrightarrow Q) \longrightarrow Q \\
\text{False} & \equiv & \forall P.\ P \\
\neg P & \equiv & P \longrightarrow \text{False} \\
P \vee Q & \equiv & \forall R.\ (P \longrightarrow R) \longrightarrow (Q \longrightarrow R) \longrightarrow R
\end{array}
$$

In conclusion

That was HOL from scratch! But we skipped some steps.

For example, what did we assume about $=$?

Anyhow, the resulting logic is consistent. (How do we know that?)

We have learned so far ...

- HOL
- Higher Order Abstract Syntax
- Defining HOL

More on HOL in COMP4161.